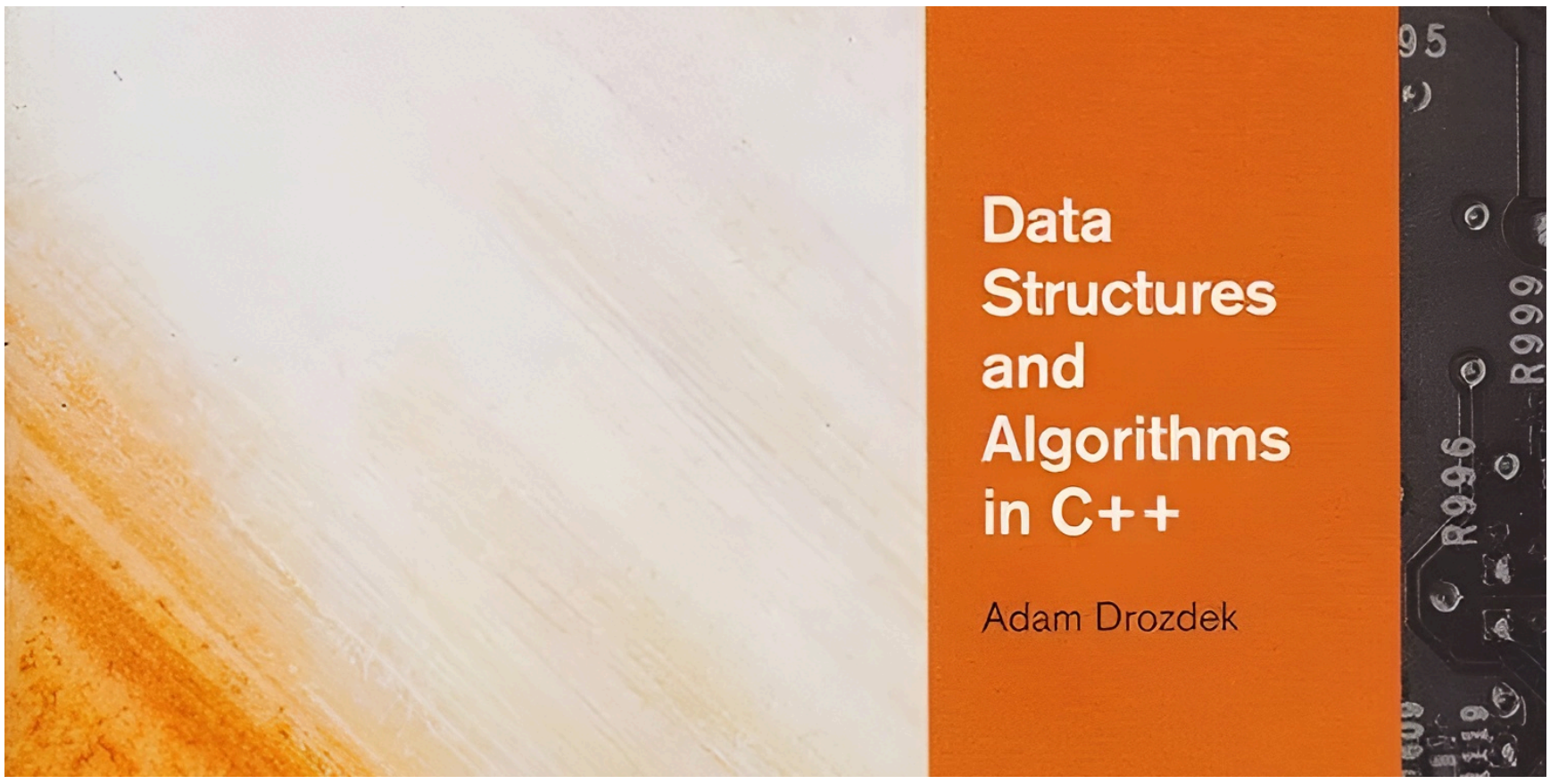
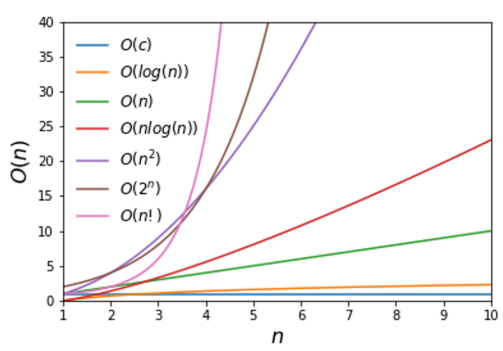


11212: Data Structures and Introduction to Algorithms

Source	https://github.com/ialbluwi/psut-data-structures

Analysis of Algorithms



- Classify Algorithms based on growth rate (ignore the coefficients)
- Consider the highest-order term when working with large input sizes.

Given a piece of code, we count the number of operations it performs. This allows us to compare the efficiency of different algorithms using the big-O notation and focusing on the best and worst cases.

```
for(int i=0; i<10; i++)  ⇒ O(1)
   Op();

for(int i=0; i<n; i++)   ⇒ O(n)
   Op();

for(int i=0; i<n; i++)
   for(int j=0; j<n; j++)  ⇒ O(n^2)
      Op();

for(int i=1; i<=n; i*=2)  ⇒ O(logn) //changing 2 changes the base of the log
      Op();
//If the inner loop depends on the outer loop ⇒ Trace and then find the sum
```

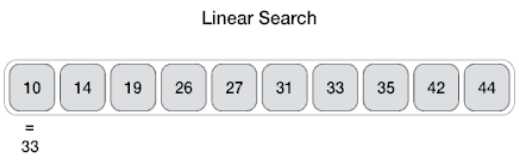
$$\text{constant} < \text{logarithmic} < \text{polynomial} < \text{exponential} < \text{factorial} < n^n$$

Searching and Sorting

Linear vs. Binary Search:

```
int search(int a[], int k, int n)
{
for (int i = 0; i < n; i++)
if (a[i] == k)
return i;

return -1;
}
```

If the array is sorted, we could put another condition to return -1 if the key is less than the current element but the worst case becomes $2n$



Best Case	1 key comparison	k is the first element
Worst Case	n comparisons	k is not in the array

```
int search(int a[], int k, int n) {
int lo = 0, hi = n-1;
while (lo <= hi) {
int mid = lo + (hi-lo) / 2;
if (k > a[mid]) lo = mid+1;
else if (k < a[mid]) hi = mid-1;
else if (k == a[mid]) return mid;
}
return -1;
}
```

Best Case	2 comparisons	k is at the mid
Worst Case	$2(\lfloor \log_2 n \rfloor + 1)$	k is less than the first element

Selection Sort :

To find the minimum and place it in its correct position (invariant). The flag helps in data moves if min is at index i.

```
void selection(int a[], int n) {
   for (int i = 0; i < n-1; i++) {
      int min_index = i;          //traverse the array; i at current

      for (int j = i+1; j < n; j++)   //look for the minimum
         if (a[j] < a[min_index])
            min_index = j;

if (i != min_index)             //swap current with minimum
      swap(a[i], a[min_index]);
}}
```

Selection Sort

5	1	7	8	6	7	5	3	0	9

Total = $O(n^2)$	Data Compares	Data Moves
Best Case	$\frac{1}{2}n(n-1)$	0
Worst Case	$\frac{1}{2}n(n-1)$	$3(n-1)$

Insertion Sort:

To insert the temporary value in an appropriate position to the portion left of that temporary value. Good with partially sorted arrays and random values.

```
void insertion(int a[], int n) {
   for (int i = 1; i < n; i++) {
      int temp = a[i];

      int j = i-1;
      while (j >= 0 && temp < a[j]) {
         a[j+1] = a[j];
         j--;
      }

   a[j+1] = temp;
}}
```

Insertion Sort

5	1	7	8	6	7	5	3	0	9

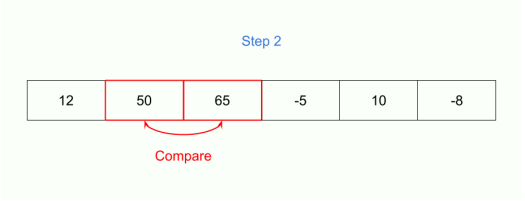
If every element is either in its correct position or only a few steps away from it, we need a few data comparisons and moves for every element.

	Data Compares	Data Moves	Total
Best Case	$(n-1)$	$2(n-1)$	$O(n)$
Worst Case	$\frac{1}{2}n(n-1)$	$\frac{1}{2}n(n-1)+2(n-1)$	$O(n^2)$

Bubble Sort:

To compare the current element with the one after it (adjacent values) and swap the values to sort it properly

```
void bubble(int a[], int n) {
   for (int i = 0; i < n-1; i++) {
   bool swapped = false;

   for (int j = n-1; j > i; j--) {
      if (a[j] < a[j-1]) {
         swap(a[j], a[j-1]);
         swapped = true;
      }}
   if (!swapped)
   break;
}}
```



The flag reduces the number of iterations to only 1 in the best case.

	Data Compares	Data Moves	Total
Best Case	$(n-1)$	0	$O(n)$
Worst Case	$\frac{1}{2}n(n-1)$	$3 \times \frac{1}{2}n(n-1)$	$O(n^2)$

Lists

A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations. It consists of nodes such that each node contains a data field and a reference/pointer to the node next to it in the list. (It is preferred over array list with frequent insertion and deletion of elements)

Singly-Linked lists

▼ Implementation of singly linked lists

```
class Node {
public:
```

Doubly-Linked lists

▼ Implementation of doubly linked lists

```
template <class T>
class DLList;
```

```
    int val;
    Node* next;

    Node(int val, Node* next) {
        this->val = val;
        this->next = next;
    }
};

class List {
public:
    List();
    ~List();

    Node* get_head() const;
    Node* get_tail() const;
    bool is_empty() const;

    bool contains(int val) const;
    int get_at(int index) const;

    void add_to_head(int val);
    void add_to_tail(int val);

    void remove_head();
    void remove_tail();
    bool remove(int val);

    void print() const;

private:
    Node* head;
    Node* tail;
};

List::List()
{
    head = nullptr;
    tail = nullptr;
}
List::~List()
{
    Node* curr = head;
    Node* del_node;

    while (curr != nullptr) {
        del_node = curr;
        curr = curr->next;
        delete del_node;
    }
}

bool List::is_empty() const

    return head == nullptr;
}

Node* List::get_head() const
{
    return head;
}

Node* List::get_tail() const
{
    return tail;
}

// --- Asymptotic complexity: O(1)
void List::add_to_head(int val)
{
    Node* new_node = new Node(val, head);

    head = new_node;
    if (tail == nullptr)
        tail = new_node;
}

// --- Asymptotic complexity: O(1)
void List::add_to_tail(int val)
{
    Node* new_node = new Node(val, nullptr);

    if (tail == nullptr)
        head = new_node;
    else
        tail->next = new_node;

    tail = new_node;
```

```

template <class T>
class DLLNode {
public:
    DLLNode(const T& val, DLLNode* next, DLLNode* prev);

    T get_val() const { return val; }
    DLLNode* get_next() const { return next; }
    DLLNode* get_prev() const { return prev; }

private:
    T val;
    DLLNode* next;
    DLLNode* prev;

    friend class DLList<T>;
};

template <class T>
DLLNode<T>::DLLNode(const T& val, DLLNode* next, DLLNode* prev)
{
    this->val = val;
    this->next = next;
    this->prev = prev;
}

template <class T>
class DLList {
public:
    DLList();
    DLList(const DLList& other);
    ~DLList();

    bool is_empty() const;

    DLLNode<T>* head_node() const;
    DLLNode<T>* tail_node() const;
    T head_val() const;
    T tail_val() const;

    void add_to_head(const T& val);
    void add_to_tail(const T& val);
    void append(const DLList& otherList);

    void remove_head();
    void remove_tail();
    bool remove(const T& val);
    void clear();

    bool contains(const T& val) const;
    DLList& operator=(const DLList& other);

private:
    DLLNode<T>* head;
    DLLNode<T>* tail;
};


template <class T>
DLList<T>::DLList()
{
    head = nullptr;
    tail = nullptr;
}

template <class T>
DLList<T>::DLList(const DLList<T>& other) {
    head = nullptr;
    tail = nullptr;
    append(other);
}

// --- Asymptotic complexity: O(n).
template <class T>
DLList<T>::~DLList()
{
    clear();
}

// --- Asymptotic complexity: O(n).
template <class T>
void DLList<T>::clear()
{
    DLLNode<T>* tmp = head;
    DLLNode<T>* del_node;

    while (tmp != nullptr) {
        del_node = tmp;
```

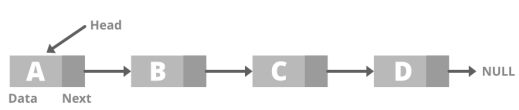
```
}

// --- Asymptotic complexity: O(1)
void List::remove_head()
{
    if (is_empty())
        return;

    Node* temp = head;
    head = head->next;
    delete temp;

    if (head == nullptr)
        tail = nullptr;
}

// --- Asymptotic complexity: O(n)
void List::remove_tail()

    if (head == tail) {
        remove_head();
        return;
    }


    Node* prev = head;
    while (prev->next != tail)
        prev = prev->next;



    delete tail;
    tail = prev;
    tail->next = nullptr;
}

// --- Asymptotic complexity:
//       * Best Case:    O(1) If the value is at the head.
//       * Worst Case:   O(n) If the value is at the node before the tail.
bool List::remove(int val)
{
    if (is_empty())
        return false;

    if (val == head->val) {
        remove_head();
        return true;
    }

    if (val == tail->val) {
        remove_tail();
        return true;
    }

    Node* node = head->next;
    Node* pred = head;

    while (node != nullptr) {
        if (node->val == val) {
            pred->next = node->next;
            delete node;
            return true;
        }

        node = node->next;
        pred = pred->next;
    }

    return false;
}

// --- Asymptotic complexity:
//       * Best Case: O(1) If val is at the head node.
//       * Worst Case: O(n) If val is not in the list.
bool List::contains(int val) const
{
    for (Node* curr = head; curr != nullptr; curr = curr->next)
        if (curr->val == val)
            return true;

    return false;
}

// --- Asymptotic complexity:
//       * Best Case: O(1) If the given index is 0.
//       * Worst Case: O(n) If the given index is the last index in the list.
int List::get_at(int index) const {
    if (is_empty() || index < 0)
        throw string("ERROR: Invalid argument in function get_at(int)");
```

```
        tmp = tmp->next;
        delete del_node;
    }

    head = nullptr;
    tail = nullptr;
}

template <class T>
bool DLList<T>::is_empty() const
{
    return head == nullptr;
}

template <class T>
DLLNode<T>* DLList<T>::head_node() const
{
    return head;
}

template <class T>
DLLNode<T>* DLList<T>::tail_node() const
{
    return tail;
}

template <class T>
T DLList<T>::head_val() const
{
    if (is_empty())
        throw string("ERROR: Attempting to retrieve a value of the head from
an empty list.");

    return head->val;
}

template <class T>
T DLList<T>::tail_val() const
{
    if (is_empty())
        throw string("ERROR: Attempting to retrieve a value of the tail from an
empty list.");

    return tail->val;
}

// --- Asymptotic complexity: O(1)
template <class T>
void DLList<T>::add_to_head(const T& val)
{

    DLLNode<T>* new_node = new DLLNode<T>(val, head, nullptr);

    if (head != nullptr)
        head->prev = new_node;

    head = new_node;
    if (tail == nullptr)
        tail = new_node;
}


// --- Asymptotic complexity: O(1)
template <class T>
void DLList<T>::add_to_tail(const T& val)
{

    DLLNode<T>* new_node = new DLLNode<T>(val, nullptr, tail);

    if (tail != nullptr)
        tail->next = new_node;


    tail = new_node;
    if (head == nullptr)
        head = new_node;
}

// --- Asymptotic complexity: O(1)
template <class T>
void DLList<T>::remove_head()
{
    if (is_empty())
        return;

    DLLNode<T>* del_node = head;
    head = del_node->next;
```

```
    int count = 0;
    Node* curr = head;

    while (count != index && curr != nullptr) {
        curr = curr->next;
        count++;
    }

    if (curr != nullptr)
        return curr->val;
    else
        throw string("ERROR: Invalid argument in function get_at(int)");
}

//using the following format: [item1, item2, item3, ...]
void List::print() const {
    cout << "[";

    Node* curr = head;
    while (curr != nullptr) {
        cout << curr->val;

        if (curr->next != nullptr)
            cout << ", ";

        curr = curr->next;}
    cout << "]";
}
```

Add_to_Head()	Add_to_Tail()	Remove_Head()	Remove_Tail()	Contains()
$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$

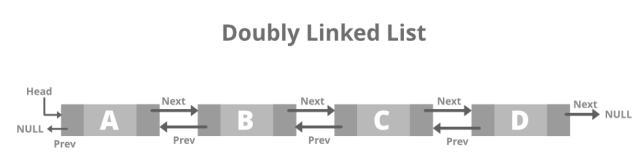
Singly Linked List




```
    if (head == nullptr)
        tail = nullptr;
    else
        head->prev = nullptr;


    delete del_node;
}

// --- Asymptotic complexity: O(1)
template <class T>
void DLList<T>::remove_tail()
{
    if (is_empty())
        return;


    DLLNode<T>* del_node = tail;
    tail = tail->prev;


    if (tail == nullptr)
        head = nullptr;
    else
        tail->next = nullptr;


    delete del_node;
}

// --- Asymptotic complexity:
//      * Best Case:   O(1) If the value is at the head or the tail.
//      * Worst Case:  O(n) If the value is at the node before the tail or if
//                     the value is not in the list.
template <class T>
bool DLList<T>::remove(const T& val)
{
    if (is_empty())
        return false;

    if (val == head->val) {
        remove_head();
        return true;
    }

    if (val == tail->val) {
        remove_tail();
        return true;
    }

    for (DLLNode<T>* curr = head->next; curr != nullptr; curr = curr->next)
{
        if (curr->val == val) {
            DLLNode<T>* pred = curr->prev;
            DLLNode<T>* succ = curr->next;
            pred->next = succ;
            succ->prev = pred;
            delete curr;
            return true;
        }
    }

    // not found
    return false;
}

// --- Asymptotic complexity: O(n).
template <class T>
bool DLList<T>::contains(const T& val) const
{
    DLLNode<T>* curr;

    for (curr = head; curr != nullptr; curr = curr->next) {
        if (curr->val == val)
            return true;
    }

    return false;
}

// --- Asymptotic complexity: O(n)
template <class T>
void DLList<T>::append(const DLList& other)
{
    if (this == &other)
        throw string("ERROR: Can't append the list to itself");
        // THINK: Why not?
        // How can we handle this case?
```

```
    DLLNode<T>* curr;

    for (curr = other.head; curr != nullptr; curr = curr->next)
        add_to_tail(curr->val);
}

template <class T>
DLList<T>& DLList<T>::operator=(const DLList<T>& other)
{
    if (this == &other)
        return *this;

    clear();
    append(other);

    return *this;
}


template<class T>
ostream& operator<<(ostream& out, const DLList<T>& list) {
    out << "[";

    DLLNode<T>* curr = list.head_node();
    while (curr != nullptr) {
        out << curr->get_val();
        if (curr->get_next() != nullptr)
            out << ", ";
        curr = curr->get_next();
    }
    out << "]";

    return out;
}
```

Add_to_Head()	Add_to_Tail()	Remove_Head()	Remove_Tail()	Contains()
$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$



A new concept was introduced in doubly linked lists implementation which is templated functions (generic programming) This allows us to create a function template whose functionality can be adapted to more than one type or class without repeating the entire code for each type.

Ordered Lists

▼ Implementation of ordered lists using doubly linked lists

```
template <class T>
class OrderedDLList;

template <class T>
class DLLNode
{
public:
    DLLNode(const T& val, DLLNode* next, DLLNode* prev);

    T get_val() const { return val; }
    DLLNode* get_next() const { return next; }
    DLLNode* get_prev() const { return prev; }

private:
    T val;
    DLLNode* next;
    DLLNode* prev;

    friend class OrderedDLList<T>;
};

template <class T>
DLLNode<T>::DLLNode(const T& val, DLLNode* next, DLLNode* prev)
{
    this->val = val;
    this->next = next;
    this->prev = prev;
}

template <class T>
class OrderedDLList {
public:
    OrderedDLList();
    OrderedDLList(const OrderedDLList<T>& other);
    ~OrderedDLList();
```

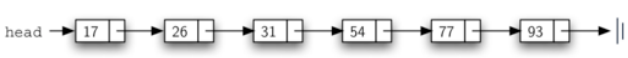
```
    bool is_empty() const;

    DLLNode<T>* head_node() const;
    DLLNode<T>* tail_node() const;

    void insert(const T& val);
    void merge(const OrderedDLList<T>& other);

    void remove_head();
    void remove_tail();
    bool remove(const T& val);
    void clear();

    bool contains(const T& val) const;
    OrderedDLList& operator=(const OrderedDLList& other);

private:
    DLLNode<T>* head;
    DLLNode<T>* tail;

    void add_to_head(const T& val);
    void add_to_tail(const T& val);
    void append(const OrderedDLList<T>& other);
};


template <class T>
OrderedDLList<T>::OrderedDLList()
{
    head = nullptr;
    tail = nullptr;
}

template <class T>
OrderedDLList<T>::OrderedDLList(const OrderedDLList<T>& other) {
    head = nullptr;
    tail = nullptr;

    append(other);
}

template <class T>
OrderedDLList<T>::~OrderedDLList()
{
    clear();
}

template <class T>
void OrderedDLList<T>::clear()
{
    while (!is_empty())
        remove_head();
}

template <class T>
bool OrderedDLList<T>::is_empty() const
{
    return head == nullptr;
}

template <class T>
void OrderedDLList<T>::add_to_head(const T& val)
{
    DLLNode<T>* new_node = new DLLNode<T>(val, head, nullptr);

    if (head != nullptr)
        head->prev = new_node;

    head = new_node;
    if (tail == nullptr)
        tail = new_node;
}

template <class T>
void OrderedDLList<T>::add_to_tail(const T& val)
{
    DLLNode<T>* new_node = new DLLNode<T>(val, nullptr, tail);

    if (tail != nullptr) //If list is non-empty
        tail->next = new_node;

    tail = new_node;
    if (head == nullptr)
        head = new_node;
}

template <class T>
```

```
void OrderedDLList<T>::remove_head()
{
    DLLNode<T>* del_node = head;

    if (is_empty())
        return;

    if (head == tail) // if only one node in the list
        head = tail = nullptr;
    else {
        head = del_node->next;
        head->prev = nullptr;
    }
    delete del_node;
}

template <class T>
void OrderedDLList<T>::remove_tail()
{
    if (is_empty())
        return;

    DLLNode<T>* del_node = tail;

    if (head == tail) // if only one node in the list
        head = tail = nullptr;
    else {
        tail = del_node->prev;
        tail->next = nullptr;
    }

    delete del_node;
}

template <class T>
bool OrderedDLList<T>::remove(const T& val)
{
    if (is_empty() || val < head->val || val > tail->val)
        return false;

    if (val == head->val) {
        remove_head();
        return true;
    }

    if (val == tail->val) {
        remove_tail();
        return true;
    }

    DLLNode<T>* curr = nullptr;
    DLLNode<T>* pred = nullptr;
    DLLNode<T>* succ = nullptr;

    for (curr = head->next; curr != nullptr; curr = curr->next) {
        if (curr->val == val) {
            pred = curr->prev;
            succ = curr->next;
            pred->next = curr->next;
            succ->prev = curr->prev;
            delete curr;
            return true;
        }

        if (val < curr->val)
            return false;
    }

    return false;
}

template <class T>
bool OrderedDLList<T>::contains(const T& val) const
{
    DLLNode<T>* curr;

    for (curr = head; curr != nullptr; curr = curr->next) {
        if (val == curr->val)
            return true;
        if (val < curr->val)
            return false;
    }

    return false;
}

template <class T>
DLLNode<T>* OrderedDLList<T>::head_node() const
```

```
{
    return head;
}

template <class T>
DLLNode<T>* OrderedDLList<T>::tail_node() const
{
    return tail;
}

template <class T>
void OrderedDLList<T>::append(const OrderedDLList& other)
{
    if (this == &other)
        throw string("ERROR: Can't append list to itself");

    DLLNode<T>* curr;
    for (curr = other.head; curr != nullptr; curr = curr->next)
        add_to_tail(curr->val);
}


// --- Asymptotic complexity:
//      * Best Case:    O(1) If the value is <= head or >= tail.
//      * Worst Case:   O(n) If the node is to be added at the node before
//                      the tail.
template <class T>
void OrderedDLList<T>::insert(const T& val)
{
    if (is_empty() || val <= head->val) {
        add_to_head(val);
        return;
    }

    if (val >= tail->val) {
        add_to_tail(val);
        return;
    }


    DLLNode<T>* curr = head->next;
    while (curr != nullptr) {
        if (val <= curr->val) {
            DLLNode<T>* pred = curr->prev;
            DLLNode<T>* new_node = new DLLNode<T>(val, curr, pred);
            curr->prev = new_node;
            pred->next = new_node;
            return;
        }
        curr = curr->next;
    }

}


// The asymptotic complexity is O(M+N)
template <class T>
void OrderedDLList<T>::merge(const OrderedDLList<T>& other)
{
    if (other.is_empty())
        return;

    if (is_empty()) {
        append(other);
        return;
    }

    OrderedDLList<T> new_list;
    DLLNode<T> *ptr1 = this->head;
    DLLNode<T> *ptr2 = other.head;

    while (ptr1 != nullptr || ptr2 != nullptr) {
        if (ptr2 == nullptr) {
            new_list.add_to_tail(ptr1->val);
            ptr1 = ptr1->next;
        }
        else if (ptr1 == nullptr) {
            new_list.add_to_tail(ptr2->val);
            ptr2 = ptr2->next;
        }
        else if (ptr1->val <= ptr2->val) {
            new_list.add_to_tail(ptr1->val);
            ptr1 = ptr1->next;
        }
        else {
            new_list.add_to_tail(ptr2->val);
            ptr2 = ptr2->next;
        }
```

```
    }

    clear();
    append(new_list);
}


// copy assignment
template <class T>
OrderedDLList<T>& OrderedDLList<T>::operator=(const OrderedDLList<T>& other)
{
    if (this == &other)
        return *this;

    clear();
    append(other);

    return *this;
}


template<class T>
ostream& operator<<(ostream& out, const OrderedDLList<T>& list) {
    out << "[";

    DLLNode<T>* curr = list.head_node();
    while (curr != nullptr) {
        out << curr->get_val();
        if (curr->get_next() != nullptr)
            out << ", ";
        curr = curr->get_next();
    }
    out << "]";

    return out;
}
```

Insert()	Merge()	Remove_Head()	Remove_Tail()	Contains()
$O(n)$	$O(m+n)$	$O(1)$	$O(1)$	$O(n)$



An ordered group of entries that can be represented with lists or arrays. Since it is ordered, data insertion is not up to the user, insertion is allowed but with specific positions.

If a program does frequent search, it is better to use an ordered array list because we can use binary search which cannot be performed in linked lists.

Stacks (LIFO)

▼ Implementation of Stacks using an array.

```
template <class T>
class StackArray
{
public:
    StackArray(int cap);
    StackArray(const StackArray& other);
    ~StackArray();

    void push(const T& val);
    T pop();
    T top() const;

    bool is_empty() const;
    bool is_full() const;

    void clear();
    StackArray& operator=(const StackArray& other);

private:
    T* data;
    int capacity;
    int last;
};


template <class T>
StackArray<T>::StackArray(int cap)
{
    if (cap <= 0)
        throw string("ERROR: Invalid capacity!");

    capacity = cap;
    data = new T[capacity];
    last = -1;
}

template <class T>
StackArray<T>& StackArray<T>::operator=(const StackArray<T>& other)
```

Queues (FIFO)

▼ Implementation of Queues using a circular array

```
const int DEFAULT_CAPACITY = 8;

template <class T>
class QueueArray
{
public:
    QueueArray();
    QueueArray(const QueueArray& other);
    ~QueueArray();

    void enqueue(const T& val);
    T dequeue();
    void clear();

    int get_size() const;
    T get_first() const;
    T get_last() const;
    bool is_empty() const;

    QueueArray& operator=(const QueueArray& other);

private:
    T* data;
    int capacity;
    int size;
    int first;
    int last;

    bool is_full() const;
    void resize(int new_cap);
};


template <class T>
QueueArray<T>::QueueArray()
{
    capacity = DEFAULT_CAPACITY;
    data = new T[capacity];
```

```
{
    if (this == &other)
        return *this;

    delete [] data;
    capacity = other.capacity;
    data = new T[capacity];

    last = other.last;
    for (int i = 0; i <= last; i++)
        data[i] = other.data[i];

    return *this;
}

template <class T>
StackArray<T>::StackArray(const StackArray<T>& other) {
    capacity = other.capacity;
    data = new T[capacity];

    last = other.last;
    for (int i = 0; i <= last; i++)
        data[i] = other.data[i];
}

template <class T>
StackArray<T>::~StackArray()
{
    delete [] data;
}

template <class T>
void StackArray<T>::push(const T& val)
{
    if (is_full())
        throw string("ERROR: Stack overflow!");

    last++;
    data[last] = val;
}

template <class T>
T StackArray<T>::pop()
{
    if (is_empty())
        throw string("ERROR: Stack underflow!");

    T val = data[last];
    --last;
    return val;
}

template <class T>
T StackArray<T>::top() const
{
    if (is_empty())
        throw string("ERROR: Attempting to retrieve an element from an empt
y stack!");

    return data[last];
}

template <class T>
bool StackArray<T>::is_empty() const
{
    return last == -1;
}

template <class T>
bool StackArray<T>::is_full() const
{
    return last == capacity - 1;
}

template <class T>
void StackArray<T>::clear()
{
    last = -1;
}
```

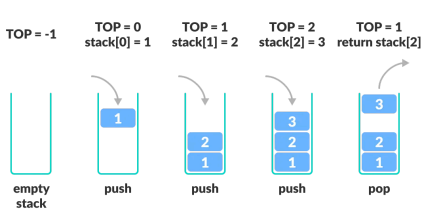
▼ Implementation of Stacks using doubly linked lists

```
#include "dll.h"

template <class T>
class StackDLL
{
public:
```

```
    last = -1;
    first = -1;
    size = 0;
}

template <class T>
QueueArray<T>::~QueueArray()
{
    delete [] data;
}

template <class T>
T QueueArray<T>::get_first() const
{
    if (is_empty())
        throw string("ERROR: An empty queue has no first element");

    return data[first];
}

template <class T>
T QueueArray<T>::get_last() const
{
    if (is_empty())
        throw string("ERROR: An empty queue has no last element");

    return data[last];
}

template <class T>
bool QueueArray<T>::is_empty() const
{
    return size == 0;
}

template <class T>
bool QueueArray<T>::is_full() const
{
    return size == capacity;
}

template <class T>
int QueueArray<T>::get_size() const
{
    return size;
}

template <class T>
void QueueArray<T>::clear()
{
    first = -1;
    last = -1;
    size = 0;

    delete [] data;
    data = new T[DEFAULT_CAPACITY];
    capacity = DEFAULT_CAPACITY;
}

template <class T>
void QueueArray<T>::enqueue(const T& val)
{
    if (is_full())
        resize(capacity * 2);

    last = (last + 1) % capacity;
    data[last] = val;
    size++;

    if (first == -1)
        first = 0;
}

template <class T>
T QueueArray<T>::dequeue()
{
    if (is_empty())
        throw string("ERROR: Can't remove from an empty queue.");

    T val = data[first];
    size--;

    if (size == 0) {
        last = -1;
        first = -1;
    }
    else
        first = (first + 1) % capacity;
```

```
    void push(const T& val);
    T pop();
    T top() const;

    bool is_empty() const;
    void clear();

private:
    DLList<T> list;
};

template <class T>
void StackDLL<T>::push(const T& val)
{
    list.add_to_tail(val);
}

template <class T>
T StackDLL<T>::pop()
{
    if (is_empty())
        throw "Stack underflow";

    T val = list.tail_val();
    list.remove_tail();
    return val;
}

template <class T>
T StackDLL<T>::top() const
{
    if (is_empty())
        throw "Stack underflow";

    return list.tail_val();
}

template <class T>
bool StackDLL<T>::is_empty() const
{
    return list.is_empty();
}

template <class T>
void StackDLL<T>::clear()
{
    list.clear();
}
```

	Doubly-Linked List	Singly-Linked List	Array List		
Push	@ Head or Tail	@ Head	@ Tail	@ Head	@ Tail
Pop	@ Head or Tail	@ Head	@ Tail	@ Head	@ Tail
	$O(1)$	$O(1)$	pop =$O(n)$	pop =$O(n)$ push =$O(n)$	$O(1)$



An abstract data type in which the last element inserted (pushed) is the first element to be deleted (popped) and only the last element has direct $O(1)$ access.

(insertion and deletion happen at the same end thus operations like `remove_tail();` in singly linked lists should be avoided as it's $O(n)$)

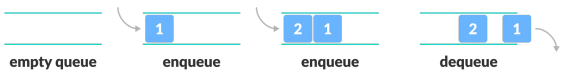
```
    if (size < capacity / 4 && capacity > DEFAULT_CAPACITY)
        resize(capacity / 2);

    return val;
}

template <class T>
QueueArray<T>::QueueArray(const QueueArray& other)
{
    capacity = other.capacity;
    size     = other.size;
    first    = other.first;
    last     = other.last;
    data     = new T[capacity];

    int j = other.first;
    for (int i = 0; i < size; i++) {
        data[j] = other.data[j];
        j = (j + 1) % other.capacity;
    }
}


template <class T>
QueueArray<T>& QueueArray<T>::operator=(const QueueArray<T>& oth
er)
{
    if (this == &other)
        return *this;

    delete [] data;

    capacity = other.capacity;
    size     = other.size;
    first    = other.first;
    last     = other.last;
    data     = new T[capacity];

    int j = other.first;
    for (int i = 0; i < size; i++) {
        data[j] = other.data[j];
        j = (j + 1) % other.capacity;
    }

    return *this;
}


template <class T>
void QueueArray<T>::resize(int new_cap)
{
    if (new_cap < size)
        throw string("ERROR: Invalid new capacity");

    T* new_data = new T[new_cap];
    int j = first;
    for (int i = 0; i < size; i++) {
        new_data[i] = data[j];
        j = (j + 1) % capacity;
    }

    delete [] data;
    data = new_data;
    capacity = new_cap;

    if (size == 0) {
        first = -1;
        last  = -1;
    } else {
        first = 0;
        last  = size - 1;
    }
}
```

▼ Implementation of Queues using doubly linked lists

```
#include "dll.h"

template <class T>
class QueueDLL
{
public:
    void enqueue(const T& val);
    T dequeue();

    T get_first() const;
    T get_last() const;
```

```
    bool is_empty() const;
    void clear();

private:
    DLList<T> list;
};

template <class T>
void QueueDLL<T>::enqueue(const T& val)
{
    list.add_to_tail(val);
}


template <class T>
T QueueDLL<T>::dequeue()
{
    T val = list.head_val();
    list.remove_head();
    return val;
}


template <class T>
T QueueDLL<T>::get_first() const
{
    return list.head_val();
}


template <class T>
T QueueDLL<T>::get_last() const
{
    return list.tail_val();
}


template <class T>
bool QueueDLL<T>::is_empty() const
{
    return list.is_empty();
}


template <class T>
void QueueDLL<T>::clear()
{
    list.clear();
}
```

	Doubly-Linked List	Singly-Linked List		Array List	
Enqueue	@ Head or Tail	@ Head	@ Tail	@ Head	@
Dequeue	@ Tail or Head	@ Tail	@ Head	@ Tail	@
	$O(1)$	dequeue=$O(n)$	$O(1)$	enqueue=$O(n)$	de



An abstract data type in which the first element inserted (enqueued) is the first element to be deleted (dequeued) and only the first and last elements have direct $O(1)$ access.

(insertion and deletion happen at opposite ends thus it is important to use a data structure that is efficient with both)

Binary Search Trees (Sets)

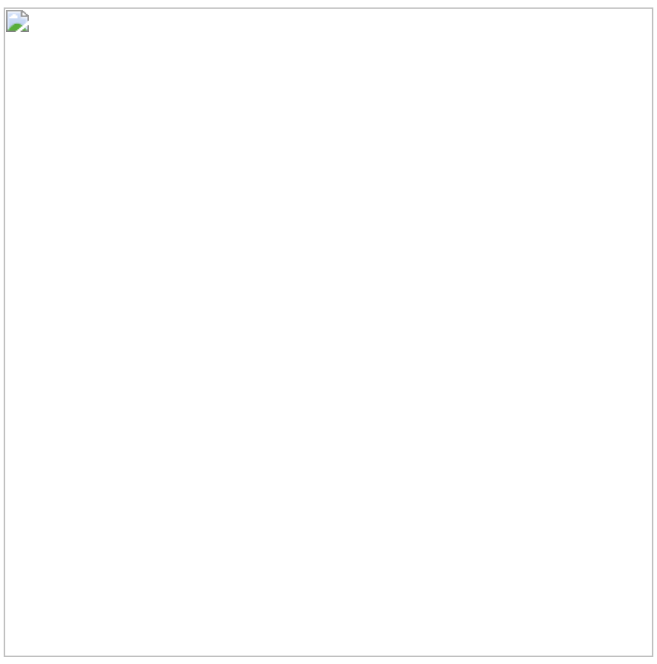
The goal is to achieve efficient search and insertion/removal for a set ADT (contains, remove and insert) implemented using a binary search tree.

A binary tree is either null or a node that is connected to at most two disjoint binary trees.

The depth of a node is the level it is in. The height is the max. number of nodes below it.

▼ Implementation of a Set using Binary Search Trees

```
template <class T>
class BST;

template <class T>
class BSTNode
{
public:
   BSTNode(const T& val, BSTNode* left, BSTNode* right);
   T get_val() const { return val; }
   BSTNode* get_left() const { return left; }
   BSTNode* get_right() const { return right; }

private:
   T val;
   BSTNode* left;
   BSTNode* right;

   friend class BST<T>;
};

template <class T>
BSTNode<T>::BSTNode(const T& val,
               BSTNode* left,
               BSTNode* right)
{
   this->val = val;
   this->left = left;
   this->right = right;
}

template <class T>
class BST
{
public:
   BST();
   BST(const BST& other);
   ~BST();

   bool is_empty() const;
   bool contains(const T& val) const;

   T max() const;
   T min() const;
   T remove_max();
   T remove_min();

   void insert(const T& val);
   bool remove(const T& val);
   void clear();

   DLList<T> elements() const;
   DLList<T> elements_level_ordered() const;
   BSTNode<T>* get_root() const { return root; }

   BST& operator=(const BST& other);

private:
   BSTNode<T> *root;

   void copy_from(BSTNode<T>* node);
   void elements(DLList<T>& result, BSTNode<T>* node) const;
   bool contains(const T& val, BSTNode<T>* node) const;

   void remove_1(BSTNode<T>* ptr, BSTNode<T>* prev);
   void remove_2(BSTNode<T>* node);

   void clear(BSTNode<T>* node);
};



template <class T>
BST<T>::BST()
{
   root = nullptr;
}

// ---- Asymptotic Complexity:
//     The running time of this function is O(n * height)
//     since n nodes are inserted:
//       * Best case: O(nlogn) if the tree is balanced.
```



Binary search tree allows us to perform binary search, all values to the left of a node are greater and the values to the right are smaller. Searching requires $O(height)$ comparisons.

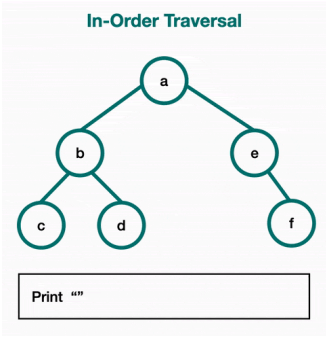
- Perfect binary search tree: all levels are full.
  $\text{Number of nodes } N = 2^{\text{height}+1} - 1$
  $\text{Height} = \log_2(N+1) - 1$
  $\text{Number of Leafs} = 2^{\text{Height}} = \frac{N+1}{2}$
  $\text{Number of internal nodes} = \frac{N-1}{2}$
- Balanced binary search tree: height of children differs by 1

(An AVL tree checks the balance after every deletion/insertion from the lowest to the highest then uses rotations to balance it)

Depth-first traversals

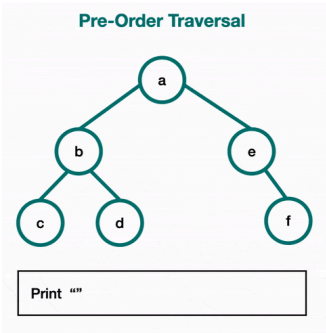
1. In-order traversal (LCR) e.g. Printing a tree

   Recursively call left then do current operation then call right.



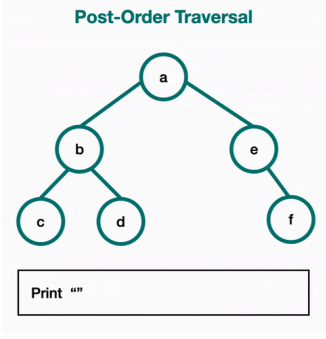
2. Pre-order traversal (CLR) e.g. Copy constructor

   Do current operation then recursively call left then right children.



3. Post-order traversal (LRC) e.g. Destructor/Clear

   Recursively call function on children then do current operation.



```
//        * Worst case: O(n^2) if the tree is a chain.
template <class T>
void BST<T>::copy_from(BSTNode<T>* node) {
    if (node == nullptr)
        return;

    insert(node->val);
    copy_from(node->left);
    copy_from(node->right);
}

template <class T>
BST<T>::BST(const BST<T>& other) {
    root = nullptr;
    copy_from(other.root);
}


template <class T>
BST<T>::~BST()
{
    clear();
}

template <class T>
bool BST<T>::is_empty() const
{
    return root == nullptr;
}


// ---- Asymptotic Complexity:
//        * Best case: O(1) If the value is at the root.
//        * Worst case: O(height), where the height is O(logn) in balanced
template <class T>
bool BST<T>::contains(const T& val) const
{
    BSTNode<T>* node = root;

    while(node != nullptr) {
        if (val == node->val)
            return true;

        if(val < node->val)
            node = node->left;
        else
            node = node->right;
    }
    return false;
}

// Recursive implementation of search.
// bool BST<T>::contains(const T& x)
// {
//      return contains(x, root);
// }

template <class T>
bool BST<T>::contains(const T& val, BSTNode<T>* node) const
{
    if (node == nullptr)
        return false;

    if (val == node->val)
        return true;

        if (val < node->val)
        return contains(val, node->left);
    else
        return contains(val, node->right);
}


// ---- Asymptotic Complexity:
//        * Best case: O(1) if the value is already in the root or if
//                  insertion happens to the left or right of the root.
//        * Worst case: O(height), where the height is O(logn) in balanced
template <class T>
void BST<T>::insert(const T& val)
{
    if (is_empty()) {
        root = new BSTNode<T>(val, nullptr, nullptr);
        return;
    }

    BSTNode<T>* curr = root;
    BSTNode<T>* prev = nullptr;
```

```
    while(curr != nullptr) {
        prev = curr;
        if (val < curr->val)
            curr = curr->left;
        else if (val > curr->val)
            curr = curr->right;
        else
            return;
    }

    BSTNode<T>* new_node = new BSTNode<T>(val, nullptr, nullptr);
    if (val < prev->val)
        prev->left = new_node;
    else
        prev->right = new_node;
}

template <class T>
DLList<T> BST<T>::elements_level_ordered() const
{
    DLList<T> result;

    if (root == nullptr)
        return result;

    QueueDLL<BSTNode<T>*> queue;
    BSTNode<T>* node = root;
    queue.enqueue(node);

    while (!queue.is_empty()) {
        node = queue.dequeue();
        result.add_to_tail(node->val);

        if (node->left != nullptr)
            queue.enqueue(node->left);
        if (node->right != nullptr)
            queue.enqueue(node->right);
    }

    return result;
}

template <class T>
DLList<T> BST<T>::elements() const
{
    DLList<T> result;
    elements(result, root);
    return result;
}

template <class T>
void BST<T>::elements(DLList<T>& result, BSTNode<T>* node) const
{
    if (node == nullptr)
        return;

    elements(result, node->left);

    result.add_to_tail(node->val);

    elements(result, node->right);
}

template <class T>
bool BST<T>::remove(const T& val)
{
    BSTNode<T>* node = root;
    BSTNode<T>* prev = nullptr;
    while (node != nullptr) {
        if (node->val == val)
            break;

        prev = node;
        if (val < node->val)
            node = node->left;
        else
            node = node->right;
    }


    if (node == nullptr)
        return false;


    if (node->left == nullptr || node->right == nullptr)
        remove_1(node, prev);
    else
        remove_2(node);
```

```
    return true;
}

template <class T>
void BST<T>::remove_1(BSTNode<T>* ptr, BSTNode<T>* prev)
{

    if (ptr == root) {
        if (root->left != nullptr)
            root = root->left;
        else
            root = root->right;

    }
    else if (ptr == prev->left) {
        if (ptr->right != nullptr)
            prev->left = ptr->right;
        else
            prev->left = ptr->left;

    }     else {
        if (ptr->right != nullptr)
            prev->right = ptr->right;
        else
            prev->right = ptr->left;
    }

    delete ptr;
}

template <class T>
void BST<T>::remove_2(BSTNode<T>* node)

    BSTNode<T>* rep = node->left;
    BSTNode<T>* prev = node;

    while (rep->right != nullptr) {
        prev = rep;
        rep = rep->right;
    }

    node->val = rep->val;
    remove_1(rep, prev);
}


template <class T>
void BST<T>::clear()
{
    clear(root);
    root = nullptr;
template <class T>
void BST<T>::clear(BSTNode<T>* node)
{
    if (node == nullptr)
        return;

    clear(node->left);
    clear(node->right);

    delete node;
}

// ---- Asymptotic Complexity:
//         * Best Case: O(1) if the root is the maximum.
//         * Worst Case: O(n) if the tree is a right-leaning chain.
template <class T>
T BST<T>::max() const {
    if (is_empty())
        throw string("ERROR: Attempting to retrieve the max value in an empt
y tree");

    BSTNode<T>* curr = root;
    while (curr->right != nullptr)
        curr = curr->right;

    return curr->val;
}
// ---- Asymptotic Complexity:
//         * Best Case: O(1) if the root is the minimum.
//         * Worst Case: O(n) if the tree is a left-leaning chain.
template <class T>
T BST<T>::min() const {
    if (is_empty())
        throw string("ERROR: Attempting to retrieve the min value in an empt
y tree");
```

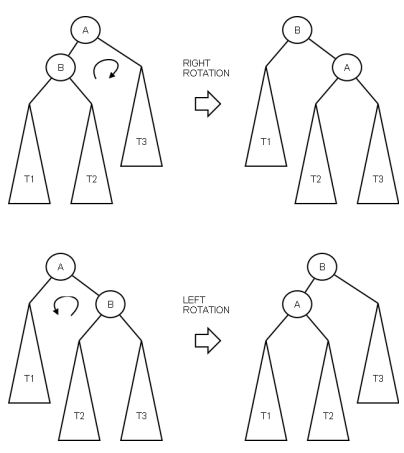
```
    BSTNode<T>* curr = root;
    while (curr->left != nullptr)
        curr = curr->left;

    return curr->val;
}

template <class T>
T BST<T>::remove_max() {
    if (is_empty())
        throw string("ERROR: Attempting to remove the max value from an e
mpty tree");

    BSTNode<T>* curr = root;
    BSTNode<T>* prev = nullptr;
    while (curr->right != nullptr) {
        prev = curr;
        curr = curr->right;
    }

    T val = curr->val;
    remove_1(curr, prev);
    return val;
}

template <class T>
T BST<T>::remove_min() {
    if (is_empty())
        throw string("ERROR: Attempting to remove the min value from an em
pty tree");

    BSTNode<T>* curr = root;
    BSTNode<T>* prev = nullptr;
    while (curr->left != nullptr) {
        prev = curr;
        curr = curr->left;
    }

    T val = curr->val;
    remove_1(curr, prev);
    return val;
}

template <class T>
BST<T>& BST<T>::operator=(const BST<T>& other)
{
    if (this == &other)
        return *this;

    clear();
    copy_from(other.root);

    return *this;
}
```

	Insert(val)	Remove(val)	Contains(val)
(Un/Ordered) List	$O(n)$	$O(n)$	$O(n)$
Unordered Array	$O(n)$	$O(n)$	$O(n)$
Ordered Array	$O(n)$	$O(n)$	$O(logn)$
(Balanced) BST !! <3	$O(logn)$	$O(logn)$	$O(logn)$

Balanced Binary Search Trees:

→ If we want to create a balanced binary tree with n elements; you either recursively insert the median to the left and right side or shuffle the elements randomly and insert them.

We also need to guarantee the tree being balanced after insertion and/or deletion using rotations!

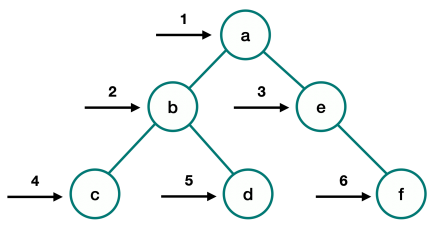


1. Left heavy left heavy: bf= 2&1 ⇒ right rotation
2. Right heavy right heavy: bf= -2&-1 ⇒ left rotation
3. Bf= -2&1 or 2&-1 ⇒ double rotations

Traversals:

Breadth-first Traversals

Maintain a queue of the nodes yet to be visited and repeat until the queue is empty (remove a node → add its children)

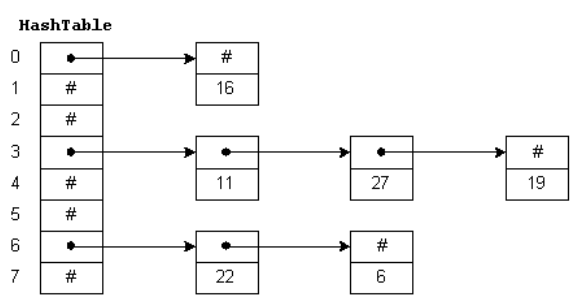


Implementation of a level-order traversal in a function

```
template <class T>
void BST<T>::level_order() const {
if (is_empty()) return;

Queue< Node<T>* > queue;
queue.enqueue(root);

while (!queue.is_empty()) {
Node<T>* node = queue.dequeue();
//operation

if (node→left != nullptr) queue.enqueue(node→left);
if (node→right != nullptr) queue.enqueue(node→right);
}
}
```

(Right-to-left BFT enqueue the right child first then the left child)

Hash Tables (Sets)



Hash function for strings (positional system):

```
int hash_value(string & str) {
int sum=0, R=1;

for (int i=str.length()-1; i>=0; i--) {
sum += R*str[i];
R *= 26;
}
return sum % m;
}
```

Using ASCII for the first character results in limited chains and non-uniform distribution.

Using the sum of ASCII values of each character results in the same sum for different strings and some values are more likely to occur.

▼ Implementation of a hash table

```
const int DEFAULT_TABLE_SIZE = 15;

template <class T>
class HashTable {
public:
   HashTable();
   HashTable(const HashTable& other);

   ~HashTable();
   bool is_empty() const;

   bool contains(const T&  val) const;
   void insert(const T&  val);
   bool remove(const T&  val);
   void clear();
   DLList<T> elements() const;

   HashTable& operator=(const HashTable& other);

private:
   DLList<T>* table;
   int n;
   int m;

   int hash_value(const T &val) const;
   void resize(int new_size);
};



template <class T>
HashTable<T>::HashTable()
{
   n = 0;
   m = DEFAULT_TABLE_SIZE;
   table = new DLList<T>[m];
}

template <class T>
HashTable<T>::HashTable(const HashTable<T>& other)
{
   this→n = other.n;
   this→m = other.m;
   table = new DLList<T>[m];

   for (int i = 0; i < m; i++)
      table[i] = other.table[i];

}
```

```
template <class T>
HashTable<T>::~HashTable()
{
   delete [] table;
}


template <class T>
bool HashTable<T>::is_empty() const
{
   return n == 0;
}


// Asymptotic Complexity:
//     - O(1) if hashing is perfect
//     - O(n) if hashing is the worst possible
template <class T>
bool HashTable<T>::contains(const T&  val) const
{
    int index = hash_value(val);
    return table[index].contains(val);
}

template <class T>
void HashTable<T>::insert(const T&  val)
{
    int index = hash_value(val);
    table[index].add_to_tail(val); // or add_to_head()

    n++;
    if (n / m >= 8)
        resize(m * 2);
}

// Asymptotic Complexity:
//     - O(n) if hashing is the worst possible
template <class T>
bool HashTable<T>::remove(const T&  val)
{
    int index = hash_value(val);
    bool deleted = table[index].remove(val);

    if (deleted) {
        n--;
        if (n / m < 2 && m > DEFAULT_TABLE_SIZE)
            resize(m / 2);
    }

    return deleted;
}

template <>
int HashTable<int>::hash_value(const int& val) const
{
    return abs(val) % m;
}

template <>
int HashTable<string>::hash_value(const string& val) const
{
    int sum = 0;
    const int R = 33;

    for (int i = 0; i < val.length(); i++)
        sum = (sum*R + val[i]) % m;

    return abs(sum);
}
// Runs in O(n + m)
template <class T>
DLList<T> HashTable<T>::elements() const
{
    DLList<T> result;

    for (int i = 0; i < m; i++)
        result.append(table[i]);

    return result;
}

template <class T>
void HashTable<T>::resize(int new_size)
{
    // O(n + m)
    DLList<T> values = elements();
    delete [] table;
```

```
    // O(m)
    m = new_size;
    table = new DLList<T>[m];

    while (!values.is_empty()) {
        int index = hash_value(values.head_val());
        table[index].add_to_tail(values.head_val());
        values.remove_head();
    }
}

template <class T>
void HashTable<T>::clear()
{
    delete [] table;
    n = 0;
    m = DEFAULT_TABLE_SIZE;
    table = new DLList<T>[m];
}

template <class T>
HashTable<T>& HashTable<T>::operator=(const HashTable<T>& other)
{
    if (this == &other)
        return *this;

    n = other.n;
    m = other.m;

    delete [] table;
    table = new DLList<T>[m];

    for (int i = 0; i < m; i++)
        table[i] = other.table[i];

    return *this;
}
```

A hash table uses a hash function to compute an index that maps to a specific value. Ideally, the mapping should be unique (i.e. perfect hashing). However, in real-life applications, the table size will depend on the range of possible values.

Collision resolution:

There will be a certain number of collisions that could be resolved using separate chaining/closed-addressing (each cell in the table could hold more than one value)

Implementation using linked list chaining:

- Insert(x) ⇒ `table[h(x)].add_to_tail(x);`
- Remove(x) ⇒ `table[h(x)].remove(x);`
- Search(x) ⇒ `return table[h(x)].find(x);`

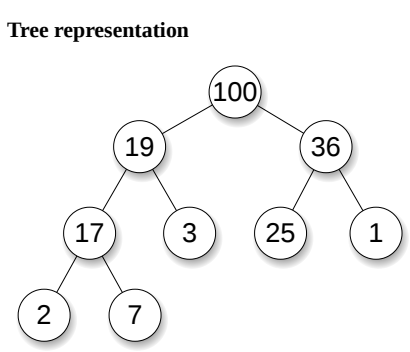
To ensure efficient operations, we introduce the load factor n/m and two assumptions:

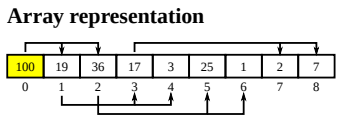
1. The elements are uniformly distributed in the table (search/remove run in $O(n/m)$)
2. n is not much larger/smaller than m (n/m is a small constant) $O(n/m) = O(1)$

   (resize the table up/down to keep m around $n/4$)

	Insert(val)	Remove(val)	Search(val)
Balanced BST	$O(logn)$	$O(logn)$	$O(logn)$
Hash Table (Average Case)	$O(1)$	$O(1)$	$O(1)$
Hash Table (Worst Case)	$O(1)$	$O(n)$	$O(n)$

Priority Queues





A priority queue is an extension of a queue, each element has a priority associated with it. The highest priority (max vs. min) is dequeued first. It is implemented with the array representation of **a binary heap.**

$$\text{Parent of i} = (i-1)/2$$

$$\text{Left child of i} = 2 * i + 1$$

$$\text{Right Child of i} = 2 * i + 2$$

Insertion

→ Insert respecting the structure (incrementing the last position in the array is equivalent to inserting left to right) then fix the order (swim) and it is logarithmic in the worst case!

There are two main components of implementing a binary heap:

1. Structure:

   It must be a complete binary tree (every level is full & insertion in the last level is left to right)

2. Order:

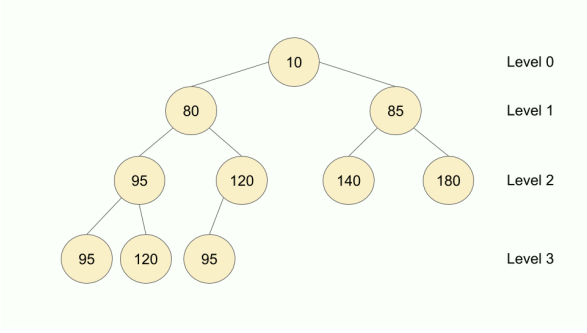
   Every node is not less than its children

⇒ The array representation uses level-order traversal on the complete binary tree.

A better selection sort:

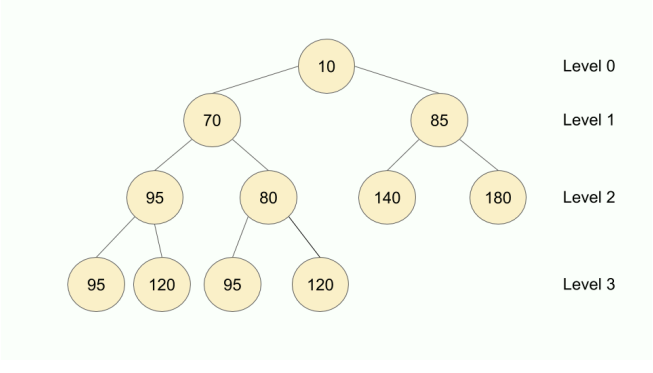
```
void heap_selection(T a[], int size){
      MaxHeap<T> heap;
      for(int i=0; i<size; i++)
         heap.insert(a[i]);

    for(int i=n-1; i>=0; i--){
         a[i]=heap.getMax();
         heap.removeMax();
}
}
```

whose running time is $O(nlogn)$ unlike selection sort's worst case of $O(n^2)$.



Deletion

→ To remove the maximum or the minimum, we swap the root with the last element which is easy to remove then we preserve the order with the sink operation which is also logarithmic in the worst case.



	Insert(val)	Remove(val)	Search(val)
Balanced BST	$O(logn)$	$O(logn)$	$O(logn)$
Binary Heap	$O(logn)$	$O(logn)$	$O(1)$

▼ C++ Standard Template Library

```
/*
 * Required Operations:
 * --------------------
 *
 * A) Stacks:
 *       1. push
 *       2. pop
 *       3. top
 *       4.    empty
 *       5. size
 *
 * B) Queues:
 *    1. push
 *    2. pop
 *    3. back
 *    4. front
 *    5. empty
 *    6. size
 *
 * C) Lists:
 *    1. push_back
 *    2. pop_back
 *    3. push_front
 *    4. pop_front
 *    5. back
 *    6. front
 *    7.     empty
 *    8. size
 *    9. clear
 *
 * D)    Vectors:
 *    1. push_back
 *    2. pop_back
 *    3. empty
 *    4. size
 *    5. back
 *    6. front
 *    7. clear
 *    8. access elements using []
 *
 * E) Iterators:
 *    1. Define a list iterator.
 *    2. Move the list iterator forward and backward using ++ and --.
 *    3. Access the value pointed to by the iterator using *
 *
 *    4. Define a vector iterator.
 *    5. Move the vector iterator forward and backward using ++, --, +=, -=.
 *    6. Access the values using * or *(iterator +/- step).
 *
 *    7. Know how to use the two iterators: begin() and end().
 *
 *    8.  Know how to use Range-based for loops
```

```
 *
 * F) Priority Queues:
 *     1. push
 *     2. pop
 *     3. top
 *     4. empty
 *     5. size
 *
 * G)    Algorithms:
 *     1. Sort
 *     2. Find
 */


#include <stack>
#include <queue>
#include <list>
#include <vector>
#include <set>
#include <algorithm>
#include <iostream>

using namespace std;


int main() {

    // Example 1:   Insert elements into a stack and then use a queue to reverse
    //                the stack.
    cout << "Printing a reversed stack:" << endl;

    stack<int> myStack;

    for (int i=0; i<10; i++)
        myStack.push(i);

    queue<int> myQueue;

    while (!myStack.empty()) {
        int x = myStack.top();
        myStack.pop();

        myQueue.push(x);
    }

    while (!myQueue.empty()) {
        int x = myQueue.front();
        myQueue.pop();

        myStack.push(x);
    }

    while (!myStack.empty()) {
        int x = myStack.top();
        myStack.pop();

        cout << x << ", ";
    }
    //-----------------------------------------------------------------------------


    // Example 2:  Insert random numbers into a priority queue and then output
    //             them in descending order.
    cout << endl << endl << "Printing the contents of a priority queue: \n";

    priority_queue<int> pq; // define a max-priority queue
    // to define a min-priority queue use:
    // priority_queue<int, vector<int>, std::greater<int>> pq;

    for (int i=0; i<10; i++) {
        int x = rand() % 10;
        pq.push(x);
    }

    while (!pq.empty()) {
        int x = pq.top();
        pq.pop();

        cout << x << ", ";
    }
    //-----------------------------------------------------------------------------




    // Example 3:  Using lists and list iterators.
    cout << endl << endl << "Printing the contents of a list "
```

```
<< "forwards and backwards: " << endl;

list<int> myList;

// Create the following list:
// 0 1 2 3 4 5 6 7 8 9
for (int i = 0; i <= 10; i++)
    myList.push_back(i);
for (int i = 10; i >= 0; i--)
    myList.push_front(i);

// printing forward
list<int>::iterator temp;
for (temp = myList.begin(); temp != myList.end(); temp++)
    cout << *temp << ", ";
cout << endl;

// printing backward
temp--;
for (; temp != myList.begin(); temp--)
    cout << *temp << ", ";
cout << *temp << endl;

// printing using the range-based for loop:
// for (int val : myList)
//         cout << val << ", ";
// cout << endl;
//-----------------------------------------------------------------------------



// Example 4: Using vectors and vector iterators.
cout << endl << endl << "Printing the contents of a vector using four "
<< "different ways." << endl;

vector<int> myVector;

for (int i=0; i<10; i++)
    myVector.push_back(i);


vector<int>::iterator temp2;
for (temp2 = myVector.begin(); temp2 != myVector.end(); temp2++)
    cout << *temp2 << ", ";
cout << endl;


for (int i=0; i<myVector.size(); i++)
    cout << *(myVector.begin() + i) << ", ";
cout << endl;


for (int i = 0; i < myVector.size(); i++)
    cout << myVector[i] << ", ";
cout << endl;


for (int val : myVector)
        cout << val << ", ";
cout << endl;
//-----------------------------------------------------------------------------



// Example 5: Sorting a vector
sort(myVector.begin(), myVector.end());
//-----------------------------------------------------------------------------



// Example 6: Removing the duplicates from a stack.
while (!myStack.empty())
    myStack.pop();


cout << "\n\nContents of the stack reversed are: " << endl;
for (int i = 0; i < 10; i++) {
    int x = rand() % 10;
    cout << x << ", ";

    myStack.push(x);
}
```

```
    myVector.clear();
    while (!myStack.empty()) {
        int x = myStack.top();
        myStack.pop();

        if (find(myVector.begin(), myVector.end(), x) == myVector.end())
            myVector.push_back(x);
    }

    while (!myVector.empty()) {
        myStack.push(myVector.back());
        myVector.pop_back();
    }

    cout << endl << endl << "After removing duplicates: " << endl;
    while (!myStack.empty()) {
        int x = myStack.top();
        myStack.pop();

        cout << x << ", ";
    }
    //-------------------------------------------------------------------------



    // ---------------------------- OPTIONAL -------------------------------- //

    // "set"               : an AVL tree that does not allow duplicates
    // "multiset"          : an AVL tree that allows duplicates
    // "unordered_set"     : a hash table that does not allow duplicates
    // "unordered_multiset" : a hash table that allows duplicates

    // use a set to generate 25 distinct random numbers between 0 and 50
    set<int> numbers;
    while (numbers.size() < 25) {
        int x = rand() % 51;
        if (numbers.find(x) == numbers.end())
            numbers.insert(x);
    }

    cout << "\n\n25 distinct random numbers:\n";
    for (int val : numbers)
        cout << val << ", ";
    cout << endl;

    return 0;
}
```